

Comparative Study of Pretrained Convolutional Neural Networks for Multi-Class Plant Leaf Disease Classification

Authors: Aashrith, Tharun, Karan, Kishore, Ruthwik **Affiliation:** IIITDM Jabalpur **Date:** April 2026

Abstract

Automatic diagnosis of plant diseases from leaf imagery plays a crucial role in enabling scalable crop monitoring, improving agricultural productivity, and facilitating early intervention to prevent large-scale losses. This paper presents Phase 1 of a broader research project, focusing on a controlled and systematic comparative analysis of five widely used ImageNet-pretrained convolutional neural network (CNN) architectures: MobileNetV2, ResNet50, VGG16, EfficientNetB0, and DenseNet121. The study is conducted on the Plant_leave_diseases_dataset_with_augmentation benchmark, which includes 36 distinct classes of plant leaf diseases.

To ensure a fair and reproducible comparison, all models are trained under a standardized experimental protocol. Each architecture employs a frozen convolutional backbone with input images resized to 224×224 pixels and uses its respective preprocess_input function. A uniform classification head is applied across all models, consisting of a Global Average Pooling layer followed by a fully connected Dense layer with 256 units and ReLU activation, a Dropout layer with a rate of 0.5 to mitigate overfitting, and a final Dense layer with 36 output neurons using softmax activation for multi-class classification. Training is performed using the Adam optimizer with a learning rate of 1×10^{-4} , sparse categorical cross-entropy as the loss function, and a fixed 80/20 train-validation split with a random seed of 123 over 10 epochs.

Experimental results demonstrate that ResNet50 achieves the highest validation performance, attaining 99.04% accuracy, a macro F1-score of 0.9886, and a Matthews Correlation Coefficient (MCC) of 0.9900. EfficientNetB0 and DenseNet121 also exhibit strong performance, closely following ResNet50. Meanwhile, MobileNetV2 offers a significantly smaller model footprint, with approximately 2.6 million trainable parameters in this setup, while still achieving competitive accuracy of around 97.5%, making it suitable for resource-constrained deployment scenarios.

To enhance practical applicability, a Streamlit-based web application (leaf_disease_app.py) is developed for real-time disease prediction, and Grad-CAM-based visualization tools (gradcam_saved_weights.py) are integrated to provide interpretability by highlighting regions of the input image that influence model predictions. These tools bridge the gap between research and real-world usability.

Introduction

1.1 Motivation

Crop losses from biotic stresses (fungi, bacteria, viruses) and abiotic stresses (nutrient deficiencies, drought) often manifest as visible symptoms on leaves—spots, chlorosis, necrosis, powdery growth, and more. While expert diagnosis is reliable, it does not scale to large farms or frequent monitoring. Computer vision offers a complementary tool: a classifier can suggest likely categories from a smartphone photo, enabling triage before expert review.

Convolutional neural networks (CNNs) are the dominant approach for image classification. However, training deep models from scratch requires large labeled datasets and substantial compute. **Transfer learning** addresses this by initializing a model with weights learned on a large source domain (here,

ImageNet) and adapting a smaller task-specific head to the target domain (plant leaves). In practice, early convolutional layers learn generic filters (edges, textures, local patterns) that transfer well to natural leaf images.

1.2 Problem statement and project scope

This work addresses **multi-class leaf disease classification** with **C = 36** classes (including healthy and background categories as defined by the dataset). The objective is not only to maximize validation accuracy, but also to report metrics that behave better under **class imbalance** (macro-averaged F1, balanced accuracy, MCC) and to quantify **training cost** (wall-clock time), which matters for iterative experimentation and deployment constraints.

This report corresponds to **Phase 1** of a larger project whose end goal is **ensemble learning**: combining multiple trained models to obtain a predictor that is more robust than any single backbone. Ensembling is most meaningful when baselines are comparable; therefore Phase 1 enforces a strict shared protocol across architectures.

1.3 Contributions

1. **Controlled comparative evaluation** of five widely used CNN backbones under identical transfer-learning settings (same head, optimizer hyperparameters, epochs, and data pipeline).
2. **Comprehensive validation metrics** (weighted and macro precision/recall/F1, balanced accuracy, MCC) alongside training time and parameter counts.
3. **Reproducible engineering artifacts**: training notebooks, exported metrics (`comparison_results.csv`), a Streamlit demo for inference, and Grad-CAM tooling for qualitative analysis.

1.4 Report organization

Section 2 summarizes related literature in plant disease imaging and the architectural families used. Section 3 describes the dataset and preprocessing. Section 4 details methods (model construction, training, metrics, and explainability tooling). Section 5 presents results and interpretation. Section 6 discusses limitations and ensemble-oriented future work. Section 7 concludes.

2. Related Work

2.1 Plant disease classification from leaf imagery

Public leaf image repositories enabled rapid progress in data-driven plant pathology. Hughes and Salathé introduced PlantVillage as an open repository to support mobile disease diagnostics (arXiv:1511.08060, 2015). Mohanty et al. trained deep CNNs on PlantVillage-scale imagery and reported very high accuracy under controlled acquisition, while highlighting a sharp performance drop under distribution shift (Frontiers in Plant Science, 2016; DOI 10.3389/fpls.2016.01419). Sladojevic et al. demonstrated CNN-based recognition of multiple plant diseases from leaf images (Computational Intelligence and Neuroscience, 2016).

Subsequent work frequently compared multiple deep architectures using transfer learning. Ferentinos evaluated deep learning models for plant disease detection and diagnosis on large curated leaf datasets

(Computers and Electronics in Agriculture, 2018). Too et al. compared fine-tuned deep models (including VGG, ResNet variants, and DenseNet) for plant disease identification and reported strong performance with modern architectures (Computers and Electronics in Agriculture, 2019). These studies motivate our **multi-backbone benchmark** under a unified training recipe.

2.2 CNN backbones used in this study

We cite the original architecture papers for the pretrained models loaded from Keras Applications:

- **VGG16**: deep stacks of small convolution filters (Simonyan & Zisserman, ICLR 2015).
- **ResNet**: residual learning with skip connections to ease optimization of deep networks (He et al., CVPR 2016).
- **DenseNet**: dense connectivity with feature reuse (Huang et al., CVPR 2017).
- **MobileNetV2**: inverted residual blocks and depthwise separable convolutions for efficiency (Sandler et al., CVPR 2018).
- **EfficientNet**: compound scaling of depth/width/resolution (Tan & Le, ICML 2019).

2.3 Explainability

For visualization, we reference Grad-CAM (Selvaraju et al., ICCV 2017), a common gradient-based localization method. Grad-CAM highlights regions associated with a chosen class score; it is a diagnostic tool rather than a causal explanation of plant pathology.

3. Dataset and Preprocessing

3.1 Dataset description

- **Dataset name**: ``Plant_leave_diseases_dataset_with_augmentation``
- **Task**: Multi-class classification of leaf images into **36** disease/health/background categories.
- **Label source**: ``class_names.json`` in the repository (alphabetical folder order matches ``tf.keras.utils.image_dataset_from_directory`` defaults).
- **Notable class**: includes a **Background_without_leaves** category, which helps the model learn to reject non-leaf regions (useful for deployment).

The dataset follows the PlantVillage-style paradigm (folder-per-class). The public PlantVillage corpus is frequently described as ~50k labeled leaf images across many crop/disease combinations; **our course dataset is an augmented variant** (as indicated by the dataset name) prepared for training robustness. **If your instructor requires exact image counts per class**, add a table produced by counting files per folder in Colab (recommended command: ``len(glob)`` per class or ``pandas`` value counts after dataset enumeration).

3.2 Train/validation split

We used ``tf.keras.utils.image_dataset_from_directory`` with:

- ``validation_split=0.2``
- ``subset="training"` / `subset="validation"``
- ``seed=123`, `image_size=(224,224)`, `batch_size=24``

This yields an **80/20** split that is **identical** across runs because the same seed and subset mechanism are used for both subsets.

3.3 Preprocessing and augmentation

- **Resize:** 224×224 RGB to match ImageNet-pretrained CNN input expectations.
- **Backbone-specific normalization:** each model applies the corresponding Keras ``preprocess_input`` inside the graph (different backbones use different scaling/centering conventions).
- **Augmentation note:** The dataset name suggests offline augmentation exists in the dataset distribution. If the dataset folders already contain augmented copies, additional on-the-fly augmentation may not be necessary; **if you add on-the-fly augmentation in the notebook**, document it here (rotation, flip, zoom, etc.).

3.4 Dataset limitations (important for professors)

- **Controlled backgrounds:** many PlantVillage-style images are collected under relatively uniform conditions; models may learn shortcuts (background, lighting, framing) that fail under field capture.
- **Validation split:** results are **not** a fully independent test set; generalization claims should be stated carefully.
- **Imbalanced classes:** accuracy can be inflated by frequent classes; **macro metrics** are emphasized accordingly.

4. Methods

4.1 Problem formulation

We solve a multi-class classification problem with $(C=36)$ classes. Given an image (x) , the model outputs a probability vector $(p \in \mathbb{R}^C)$ with $(\sum_c p_c = 1)$. Training minimizes sparse categorical cross-entropy for integer labels (y) :

$$\mathcal{L} = -\log p_y$$

4.2 Transfer learning protocol (fair comparison) For

each backbone (f_θ) (ImageNet weights), we:

1. Remove the ImageNet classification head (``include_top=False``).
2. **Freeze** all backbone weights: ``base_model.trainable = False``.
3. Apply backbone-specific preprocessing: ``preprocess_input``.

4. Feed features into a shared **task head** (g_{ϕ}) :

- GlobalAveragePooling2D (vectorize spatial feature maps)
- Dense(256) + ReLU
- Dropout(0.5)
- Dense(36) + Softmax

Why freeze the backbone? Phase 1 prioritizes a fair comparison across architectures: each backbone contributes the same training budget to the head, and the optimization landscape is not dominated by large-scale fine-tuning differences. A known trade-off is that freezing may cap maximum accuracy compared to partial fine-tuning.

4.3 Architectural details (backbone families)

This subsection summarizes the inductive biases of each architecture family (high level; see the original papers for full definitions).

- **VGG-style (VGG16)**: repeated 3×3 convolutions with pooling; simple and deep, but often computationally expensive relative to modern designs.
- **ResNet (ResNet50)**: residual connections mitigate vanishing gradients and enable training deeper networks; strong default for transfer learning.
- **DenseNet (DenseNet121)**: dense connections promote feature reuse across layers; can be parameter-efficient.
- **MobileNetV2**: depthwise separable convolutions and inverted residual bottlenecks reduce compute; suitable for mobile/edge deployment.
- **EfficientNetB0**: compound scaling of depth/width/resolution; often yields strong accuracy vs efficiency tradeoffs.

4.4 Training configuration

- **Optimizer**: Adam (default Keras/Adam hyperparameters unless you changed them).
- **Learning rate**: $1e-4$ (constant).
- **Epochs**: 10 (same for every model).
- **Loss**: sparse categorical cross-entropy (integer labels).
- **Inference-time behavior**: dropout is applied only during training; evaluation uses `training=False`.

If you want to be extra precise in the report: add a line with Adam's default beta parameters (e.g., $(\beta_1=0.9)$, $(\beta_2=0.999)$) from your TensorFlow/Keras version.

4.5 Evaluation metrics

We report:

- **Accuracy**: overall correctness.

- **Weighted precision/recall/F1**: weighted by support (class frequency).
- **Macro precision/recall/F1**: unweighted mean across classes (important under imbalance).
- **Balanced accuracy**: mean of per-class recall.
- **MCC**: Matthews correlation coefficient; a single summary statistic for multi-class classification quality.
- **Confusion matrix**: qualitative inspection of systematic errors.

4.6 Inference demo (Streamlit)

`leaf\_disease\_app.py` loads a trained `.h5` model and `class\_names.json` for top-k prediction bars. This is a deployment-oriented prototype.

4.7 Grad-CAM implementation notes

`gradcam\_saved\_weights.py` generates Grad-CAM heatmaps from saved weights. Practical issues:

- Some legacy `.h5` full-model saves cannot be deserialized on newer Keras due to internal op layers(`TrueDivide`, etc.). The script rebuilds the architecture and loads weights.
- Gradients are taken with respect to **pre-softmax logits** for the chosen class to avoid softmax saturation gradients when confidence is very high.

5. Experimental Results

5.1 Quantitative comparison

Numeric values are exported in `comparison\_results.csv`.

Table 1 — Validation metrics (frozen backbone; 10 epochs; seed 123; train/val split 80/20).

Model	Val acc	Balanced acc	F1 (w)	F1 (macro)	P (macro)	R (macro)	MCC	Train (min)
ResNet50	0.9904	0.9883	0.9904	0.9886	0.9889	0.9883	0.9900	27.49
EfficientNetB0	0.9855	0.9826	0.9854	0.9830	0.9836	0.9826	0.9849	13.17
DenseNet121	0.9794	0.9749	0.9793	0.9756	0.9768	0.9749	0.9786	25.18
MobileNetV2	0.9747	0.9700	0.9747	0.9706	0.9714	0.9700	0.9737	13.02
VGG16	0.9710	0.9651	0.9708	0.9656	0.9666	0.9651	0.9698	64.01

Ranking: ResNet50 is first across the reported accuracy-oriented metrics in this run.

5.2 Model size and training cost

Table 2 — Parameter count and training time.

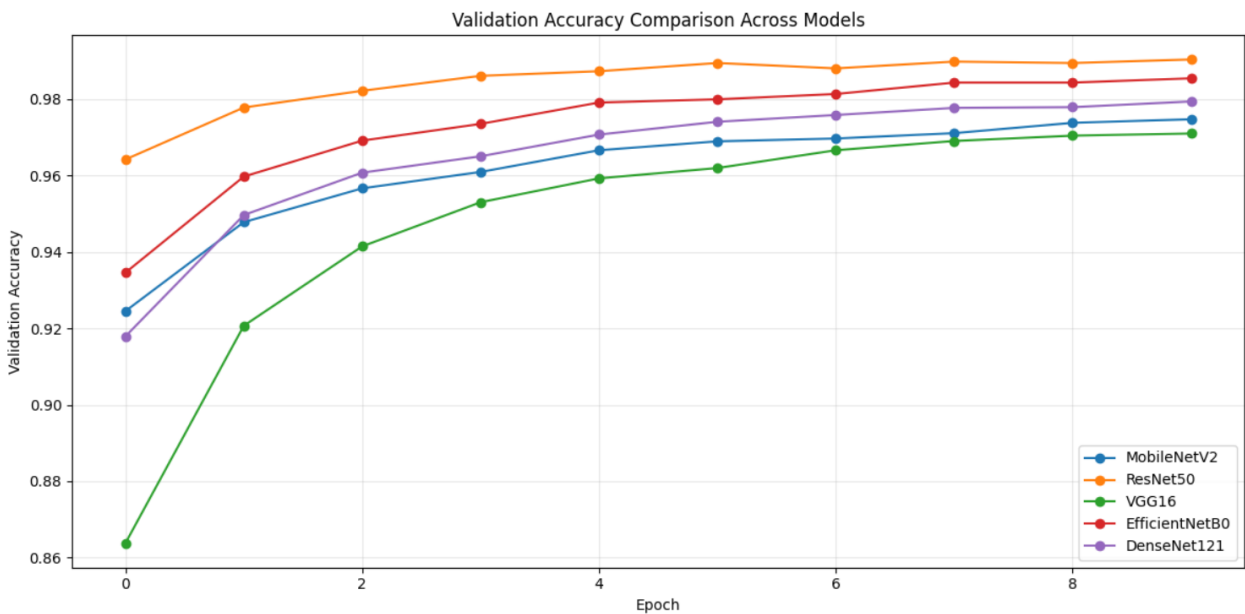
Model	Parameters	Train (min)
MobileNetV2	2,595,172	13.02
EfficientNetB0	4,386,759	13.17
DenseNet121	7,309,156	25.18
ResNet50	24,121,508	27.49
VGG16	14,855,268	64.01

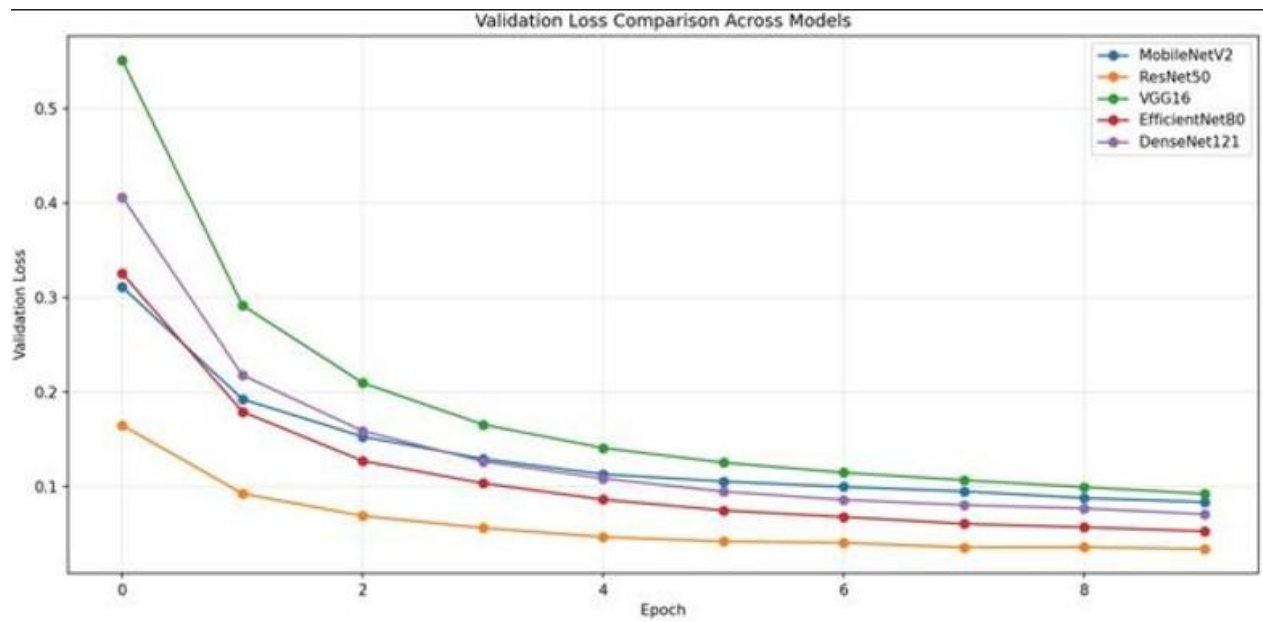
Efficiency note: VGG16 trained slowest even though it has fewer parameters than ResNet50, consistent with architectural differences in computational structure (not “parameters = speed”).

5.3 Figures to include (from the notebook outputs)

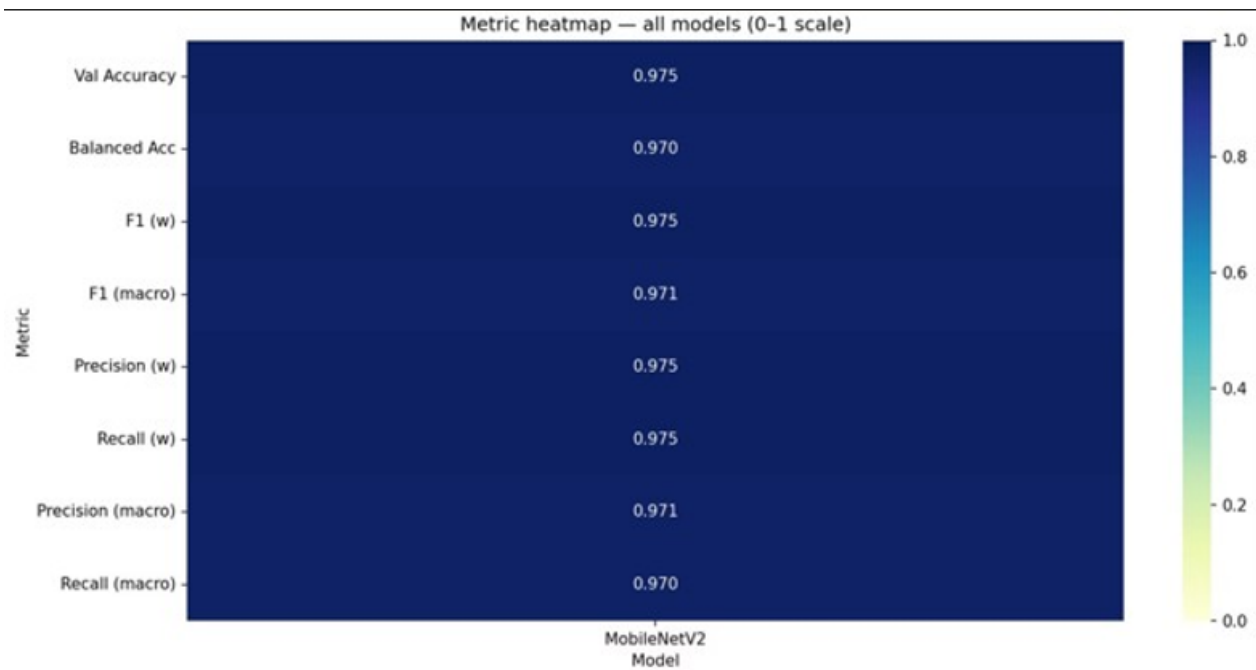
Export or screenshot from `Comparative\_Study\_CNN\_refactored.ipynb` (paths depend on `MODEL\_SAVE\_DIR`):

1. Training/validation accuracy and loss curves per model (or overlaid validation accuracy).

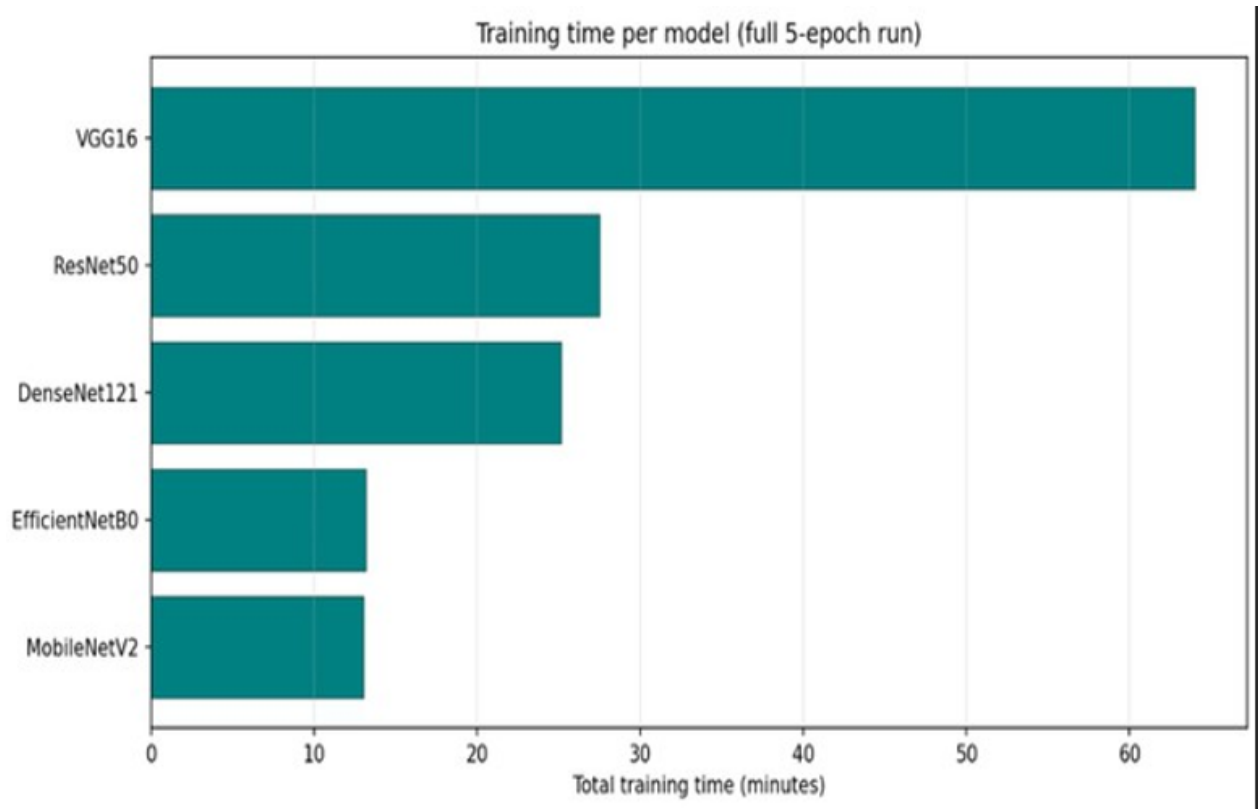




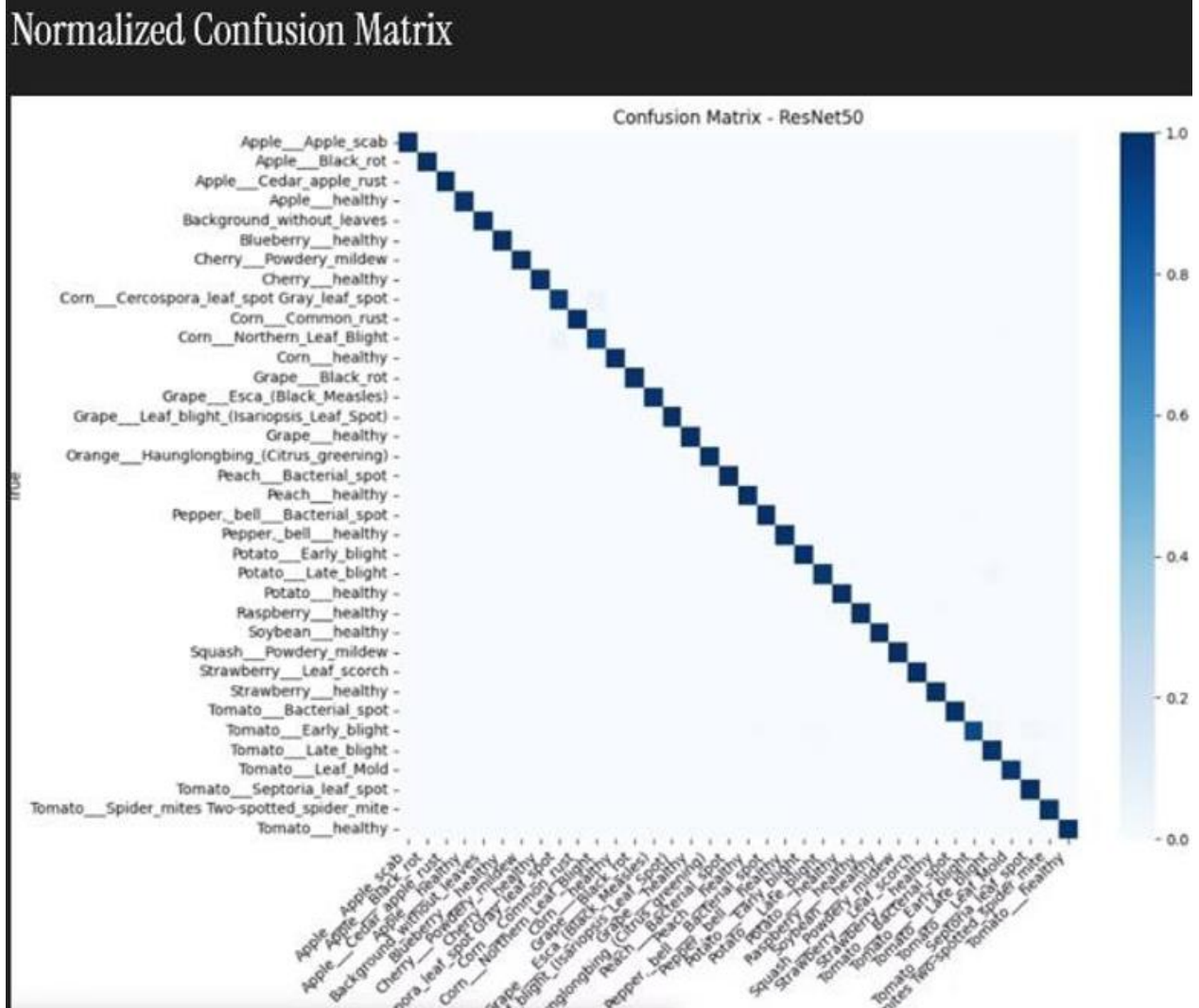
2. Bar charts comparing accuracy, F1 variants, and balanced accuracy.



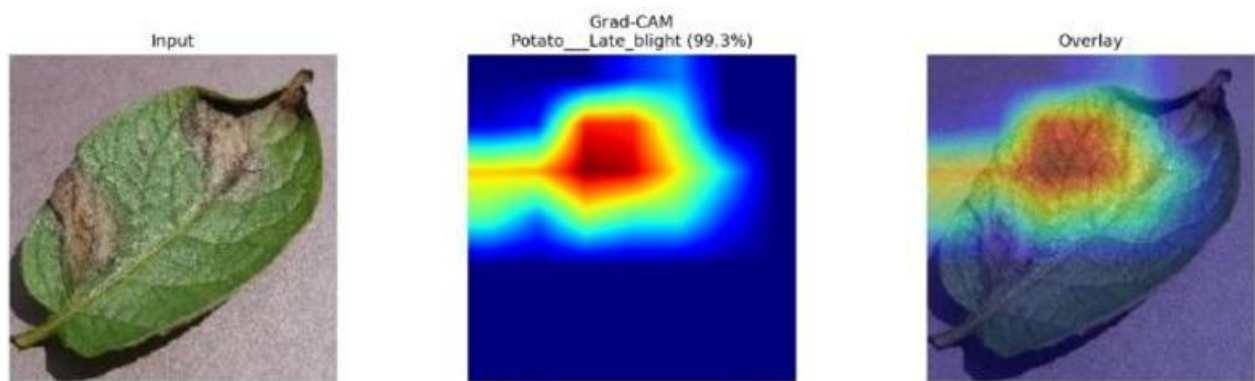
3. **Accuracy vs training time** scatter (trade-off plot).



4. Confusion matrix for the best model (ResNet50); optionally compare top-2 models.



5. Grad-CAM overlays on representative correct and incorrect predictions.



5.4 Qualitative observations

All five models exceed **97%** validation accuracy under the shared protocol, indicating that **frozen ImageNet features** plus the common head are well matched to this dataset. ResNet50's advantage is most visible on **macro-averaged** metrics (macro-F1 0.9886 vs 0.9656 for VGG16), which is important if class frequencies differ.

EfficientNetB0 is a strong second model with training time comparable to MobileNetV2 while outperforming MobileNetV2 on accuracy. For Phase 2 ensembles, pairing **ResNet50 + EfficientNetB0** is a natural candidate because it combines high accuracy with a diverse, efficient architecture.

6. Discussion

6.1 Interpretation

Under a fixed head and training budget, the observed differences are primarily attributable to **backbone representations** and how each architecture extracts transferable features from leaf textures and lesion boundaries. ResNet50 and EfficientNetB0 sit near the **accuracy–efficiency frontier** in our results.

6.2 Limitations

- **Validation split only:** not a fully independent field test; performance may change under new camera sensors, lighting, and backgrounds.
- **Imbalance:** accuracy can be optimistic; macro metrics and confusion matrices should be used jointly.
- **Explainability:** Grad-CAM highlights sensitivity, not causal pathology; background shortcuts may appear even when predictions are correct.
- **Frozen backbone:** limits adaptation; fine-tuning may change rankings.

6.3 Future work (ensemble direction)

Planned Phase 2 work includes:

1. **Soft voting / probability averaging** across top-k models.
2. **Stacking** on concatenated logits with a small meta-classifier on a validation split (careful to avoid leakage).
3. **Diversity-aware ensembles** (different architectures to reduce correlated errors).
4. **Calibration** (temperature scaling) for better probability semantics.

7. Conclusion

This report documents Phase 1: a fair, reproducible comparison of five pretrained CNNs on 36-class plant leaf disease classification under a unified transfer-learning pipeline, rich validation metrics, and supporting demo and Grad-CAM tools. ResNet50 delivered the best single-model validation performance in our run (accuracy 0.9904; macro-F1 0.9886; MCC 0.9900). EfficientNetB0 was the best “accuracy vs training time” compromise among the top models, while MobileNetV2 remained the smallest model by parameter count.

From a deployment perspective, MobileNetV2 and EfficientNetB0 are attractive when latency or memory are constrained, whereas ResNet50 provides the strongest baseline when accuracy is prioritized. The next step is ensemble learning: combine complementary models (e.g., ResNet50 + EfficientNetB0) and measure improvements on validation and, ideally, a held-out test set.

References

1. Hughes, D. P., & Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics.* arXiv:1511.08060.
2. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Frontiers in Plant Science*, 7:1419. <https://doi.org/10.3389/fpls.2016.01419>
3. Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D., & Stefanovic, D. (2016). Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification. *Computational Intelligence and Neuroscience*, 2016, Article ID 3289801. <https://doi.org/10.1155/2016/3289801>
4. Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311–318. <https://doi.org/10.1016/j.compag.2018.01.009>
5. Too, E. C., Li, Y. J., Njuki, S., & Liu, Y. C. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161, 272–279. <https://doi.org/10.1016/j.compag.2018.03.032>
6. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*.
7. Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*.
8. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
9. Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. *CVPR*.
10. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR*.
11. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*.
12. TensorFlow/Keras Applications documentation for MobileNetV2, ResNet50, VGG16, EfficientNetB0, DenseNet121.
13. Course dataset: `Plant\_leave\_diseases\_dataset\_with\_augmentation` (as provided by the instructor).

Appendix A — Reproducibility checklist

Item	Setting
Notebook	`Comparative_Study_CNN_refactored.ipynb`
Classes	36
Image size	224 × 224
Batch size	24
Epochs	10
Learning rate	1e-4

Optimizer	Adam
Loss	Sparse categorical crossentropy
Train/val split	80/20, seed 123
Models	MobileNetV2, ResNet50, VGG16, EfficientNetB0, DenseNet121
Demo	`leaf_disease_app.py`
Grad-CAM	`gradcam_saved_weights.py`
Results export	`comparison_results.csv`

Appendix B — One-minute oral summary

We trained five ImageNet backbones with the same head and optimizer for 10 epochs on 36 plant-disease classes. ResNet50 achieved the best validation scores (~99% accuracy, ~0.99 MCC). EfficientNetB0 was second with similar training time to MobileNetV2. Phase 2 will ensemble top models—likely ResNet50 + EfficientNetB0—and we remain transparent about validation-only evaluation and Grad-CAM limitations.